IMPLEMENTAION D'UNE BASE DE DONNEE POUR RESOUDRE UN PROBLEME DE SUPPORT ET DOCUMENTATION

MEMBRE : DEDZO CONSTANTIN SRI

Objectifs :

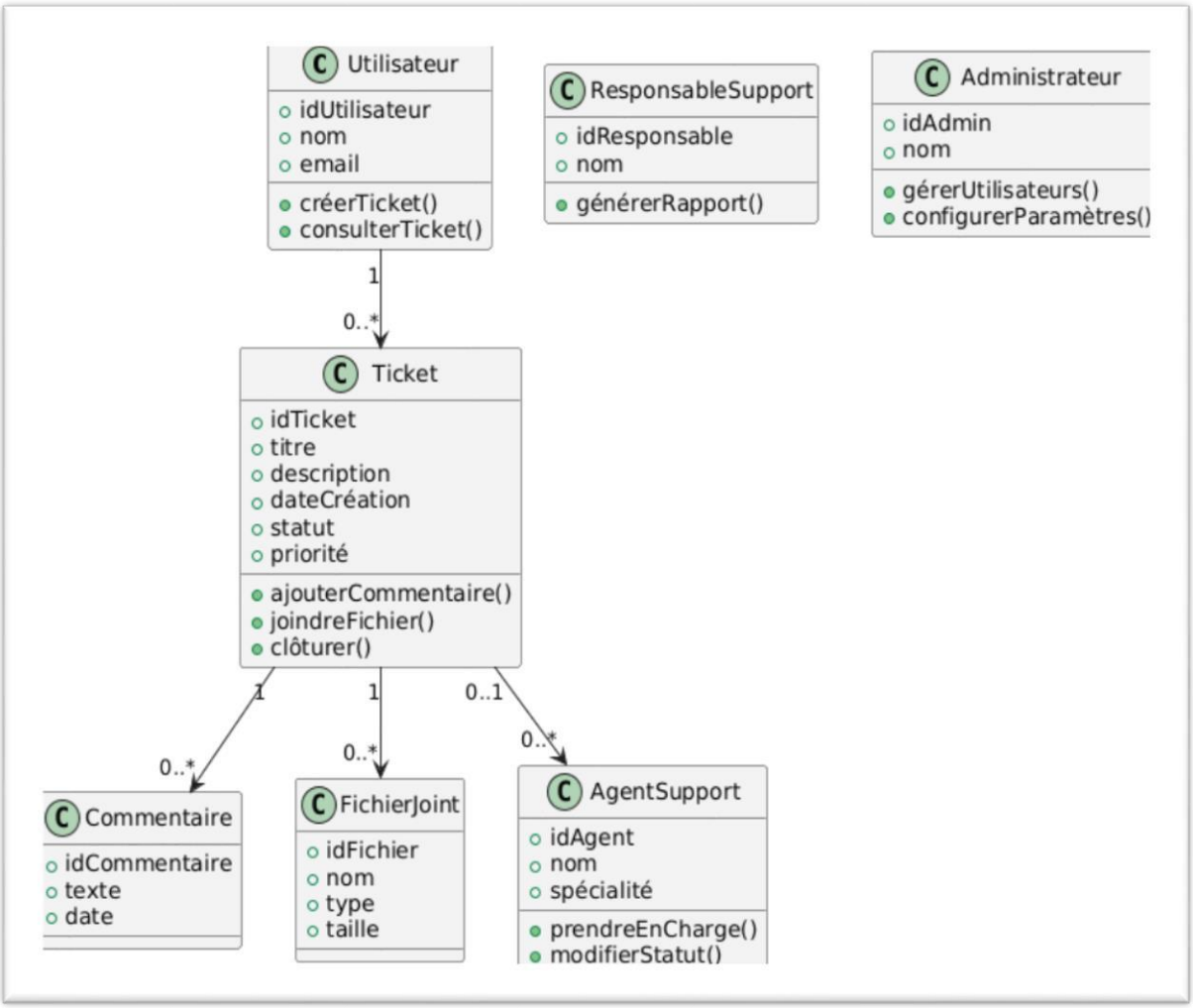
- Centraliser les tickets de support
- Assurer le suivi des incidents
- Réduire les délais de résolution
- Améliorer la traçabilité et l'historique des interventions

Étapes principales :

1. Analyse des besoins du service support
2. Conception du modèle de données (diagramme entité-relation)
3. Création des tables (utilisateurs, tickets, priorités, statuts, historiques, etc.)

. Acteurs du système :

Acteur	Rôle
Utilisateur	Soumettre des tickets, consulter l'avancement
Agent support	Traiter et résoudre les tickets
Responsable support	Analyser les performances, générer des rapports
Administrateur	Gérer utilisateurs et configuration du système



Utilisateur "1" --> "0..*" Ticket

Ticket "1" --> "0..*" Commentaire

Ticket "1" --> "0..*" FichierJoint

Ticket "0..1" --> "0..*" AgentSupport

Administrateur "1" --> "0..*" Utilisateur

Administrateur "0..1" --> "0..*" Ticket

Classes principales et relations

Classe	Attributs principaux	Méthodes principales
Utilisateur	idUtilisateur, nom, e	créerTicket(), consulterTicket()
Ticket	idTicket, titre, description, dateCréation, statut, priorité	ajouterCommentaire(), joindreFichier(), clôturer()
Commentaire	idCommenta- texte, date	-
FichierJoint	idFichier, nor type, taille	-
AgentSupport	idAgent, nom spécialité	prendreEnCharge(), modifierStatut()
ResponsableSupport	idResponsab	générerRapport()
Administrateur	idAdmin, nor	gérerUtilisateurs(), configurerParamètres()

- Les Relations et Cardinalités

Relation	Cardinalité	Explication
Utilisateur → Ticket	1 à 0..*	Un utilisateur peut créer plusieurs tickets, mais un ticket appartient à un seul utilisateur.
Ticket → Commentaire	1 à 0..*	Un ticket peut avoir plusieurs commentaires pour le suivi.
Ticket → FichierJoint	1 à 0..*	On peut attacher plusieurs fichiers à un même ticket.
AgentSupport → Ticket	0..* à 0..1	Un agent peut gérer plusieurs tickets, mais un ticket est affecté à un seul agent à la fois.

```sql
CREATE DATABASE GestionSupport;
USE GestionSupport;

CREATE TABLE Utilisateur (
    idUtilisateur INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL
);

CREATE TABLE AgentSupport (
    idAgent INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    specialite VARCHAR(100)
);

CREATE TABLE ResponsableSupport (
    idResponsable INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL
);

CREATE TABLE Administrateur (
    idAdmin INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL
);

CREATE TABLE Ticket (
    idTicket INT AUTO_INCREMENT PRIMARY KEY,
    titre VARCHAR(200) NOT NULL,
    description TEXT,
    dateCreation DATETIME DEFAULT CURRENT_TIMESTAMP,
    statut VARCHAR(50),
    priorite VARCHAR(50),
    idUtilisateur INT,
    idAgent INT,

    FOREIGN KEY (idUtilisateur) REFERENCES Utilisateur(idUtilisateur),
    FOREIGN KEY (idAgent) REFERENCES AgentSupport(idAgent)
);

CREATE TABLE Commentaire (
    idCommentaire INT AUTO_INCREMENT PRIMARY KEY,
    texte TEXT,
    date DATETIME DEFAULT CURRENT_TIMESTAMP
```

```sql
CREATE TABLE Commentaire (
    idCommentaire INT AUTO_INCREMENT PRIMARY KEY,
    texte TEXT,
    date DATETIME DEFAULT CURRENT_TIMESTAMP,
    idTicket INT,

    FOREIGN KEY (idTicket) REFERENCES Ticket(idTicket)
);

CREATE TABLE FichierJoint (
    idFichier INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(200),
    type VARCHAR(50),
    taille INT,
    idTicket INT,

    FOREIGN KEY (idTicket) REFERENCES Ticket(idTicket)
);

INSERT INTO Utilisateur (nom, email)
VALUES ('Ali', 'ali@email.com');

INSERT INTO AgentSupport (nom, specialite)
VALUES ('Sara', 'Réseau');

INSERT INTO Ticket (titre, description, statut, priorite, idUtilisateur, idAgent)
VALUES ('Problème connexion', 'Impossible de se connecter', 'Ouvert', 'Haute', 1, 1);

INSERT INTO Commentaire (texte, idTicket)
VALUES ('Nous analysons le problème', 1);
```